Binary Search & Strings

⇒ What is Binary Search?

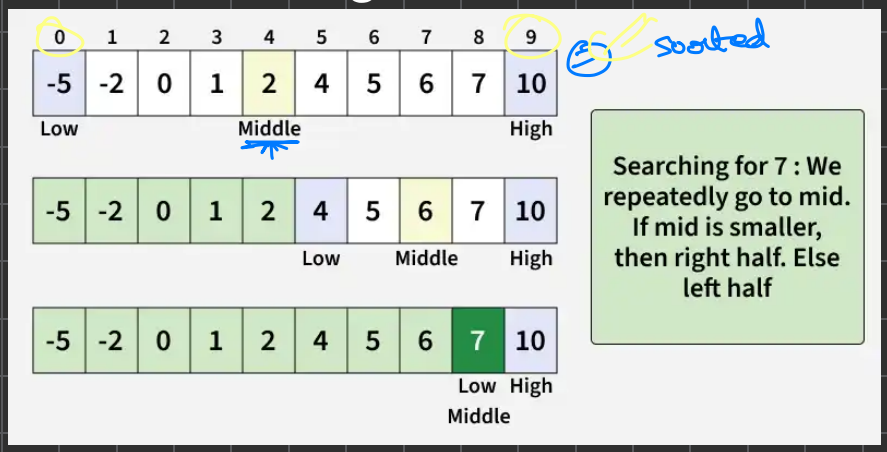

Image from GeeksForGeeks

```
binary_search(arr, t):
{ l=0, h=n-1 # n is no of elem.
  while (l<=h)
  {
      m=(l+h)//2
      if (arr[m] == t) ret m;
      if (t > arr[m])
          l = m+1;
      else
          h = m-1
  }
  return -1;
}
```

⇒ How to determine we need to apply Binary Search?

⇒ Applying two-pointers & sliding window on strings.

T.C = O(log(n))

S.C = O(1)

PROBLEM - 1

34. Find First and Last Position of Element in Sorted Array

Medium | Topics | Companies

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

Example 2:

```
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
```

Example 3:

```
Input: nums = [], target = 0
Output: [-1,-1]
```

Constraints:

- `0 <= nums.length <= 10^5`
- `-10^9 <= nums[i] <= 10^9`
- `nums` is a non-decreasing array.
- `-10^9 <= target <= 10^9`

nums = [5, 7, 7, 8, 8, 10], target = 8
(indices: 0 1 2 3 4 5)

Brute:

T.C = O(n)

S.C = O(1)

Optimal:

nums = [5, 7, 7, 8, 8, 10], target = 8
(indices: 0 1 2 3 4 5; l = 2, m = 3, h = 4)

first occurance

$m = 5 + 0 // 2 = 2$

$m = \frac{3+5}{2} = 4$

$m = \frac{3+3}{2} = 3$

first = ~~-1~~ ~~4~~ 3

T.C = O(log(n))

S.C = O(1)

nums = [1, 2, 3, 3, 3, 4], target = 3
(indices: 0 1 2 3 4 5; h = 4, ~~5~~ l)

$m = \frac{l+h}{2} = 2$

$m = \frac{3+5}{2} = 4$

$m = \frac{5+5}{2} = 5$

last = ~~-1~~ ~~2~~ 4

PROBLEM-2

3. Longest Substring Without Repeating Characters

Medium | Topics | Companies | Hint

Given a string `s`, find the length of the **longest** substring without duplicate characters.

Example 1:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

Example 3:

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 5 * 10^4`
- `s` consists of English letters, digits, symbols and spaces.

s = a b c a b c b b

a
ab
abc
abca

0 1 2 255
[ ]
len = 256

Brute force:

[a b c d e]
[0 1 2 3 4]

a b c a b c b b

max_len = 0
for i : 0 → n {
  hash[256] = {0}
  for j : i → n
  {
    if (hash[s[j]] >= 1) break;
    max_len = max(max_len, j-i+1)
    hash[s[j]] += 1
  }
}

a
ab
abc → 3
abca

b
bc
bca → 3
bcab

0 1 2 3 4 5 6 7
a b c a b c b b

(j - i + 1) = 2 - 0 + 1 = 3

T.C = O(n²)
S.C = O(256)

Optimal Approach:

[l : r]

0 1 2 3 4 5 6 7
s = a b c a b c b b

7 - 7 + 1 = 1

[l : r] } valid substring

a b c d
0 1 2 3 255
[-1, -1, -1, -1, ..., -1]
3 6 5

max_len = 0 ; (r - l + 1)
3

T.C = O(n)

[l : r]

	0	1	2	3	4	5
s =	p	w	w	k	e	w
	*		*	l		
	*	*	*	*	*	*

	p	k	w	e
[	*	*	*	*]
	0	3	2	4

max_len = 0 * 3

len = r - l + 1

	0	1	2	3
s =	a	2	a	2
	*	l		
	*	*	r	

	a	b	2
[	*	-1	*]
	0		1

max_len = 0 * 2

[l : r]

	0	1	2	3
s =	a	b	b	a
	*		l	
	*	*	*	r

	a	b
[	*	*]
	3	2

max_len = 0 * 2

PROBLEM - 3

33. Search in Rotated Sorted Array

Medium | Topics | Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

Example 3:

```
Input: nums = [1], target = 0
Output: -1
```

nums = [4, 5, 6, 7, 0, 1, 2]

Brute:

T.C = O(n)

S.C = O(1)

nums =

0	1	2	3	4	5
4	5	6	7	0	1
l		m			h

Optimal:

target = 0

0	1	2	3	m 4	5	6
4	5	6	7	0	1	2
				l		
				h		

m = 3, if nums[m] == target:

left sorted check

if nums[l] <= nums[m]:

if nums[l] <= target <= nums[m]

h = m - 1

else

l = m + 1

else

if nums[m] =< target <= nums[h]

l = m + 1

else

h = m - 1

return -1

	0	1	2	3	4	5
nums = [	4,	5,	6,	1,	2,	3],
	~~l~~		~~m~~	~~l~~	~~m~~	h
						l
						m

target = 3

$m = \frac{l+h}{2}$

$m = 2$